银盾智行

面向个人银行业务的安全可信智能体

技术文档

赛道	国内赛道 银行 AI 智能体
版本	1.0
日期	2026-09-14

文档说明

本文面向竞赛评审，说明银盾智行当前可运行版本的系统架构、核心算法、安全执行方式、数据模型、部署与测试结果。文档以仓库中的 Java、Vue、Flyway、Compose 和自动化测试为事实来源，不把尚未实现的规划写成现状。

项目已经覆盖智能转账、账单分析、理财操作、卡片管理、订阅与代扣管理、跨场景任务联动。所有演示数据均为固定规则生成的虚构数据。默认 offline-demo profile 不依赖外部模型或公网连接。

当前版本适合比赛演示和安全机制验证。它不等同于银行生产系统，也不构成合规认证、渗透测试结论或金融建议。

实现结论

评审关注点	当前实现
自然语言与 Agent	确定性意图收敛配合可插拔 BankingModelClient；用户控制的 Tool Calling Loop 最多运行
安全调用	每轮动态签发 AllowedToolSetSnapshot，模型侧最小暴露，SecureToolGateway 在执行端 次验证
高风险操作	Tool 只执行 PREPARE；独立确认、AAL、最终风险复核与事务 Service 完成真实变更
可解释性	允许和拒绝均产生 reason code、Evidence、Tool Security Audit 或 Operation/Risk Audit
可复现性	固定 dataset version、seed、baseline date 和摘要；Docker Compose 一键启动
工程验证	2026-09-11 验收记录包含 162 项后端测试、12 项前端单元测试、9 项 UI 场景和 9 项 API/Agent 验收

项目目标与业务范围

银盾智行把自然语言入口放在个人银行业务之上，同时把模型与资金、卡片、订阅和持仓的真实写入隔离。用户可以用日常表达发起任务；系统把身份、权限、风险、确认和记账留在确定性代码中。

业务场景	示例请求	执行边界
智能转账	从日常账户向张伟转账 88 元	生成待确认操作；确认前余额不变
账单分析	分析尾号 8899 账户本月账单	Java BigDecimal 计算收支、分类、商户和对比
理财操作	购买银盾稳享产品 1000 元	先完成适当性检查；购买固定需要 AAL2 和确认
卡片管理	冻结尾号 1001 的卡片	状态变化需要确认；解冻或放宽控制需要 AAL2
订阅管理	分析订阅成本并取消视频订阅	分析自动完成；取消停在确认前
跨场景联动	分析账单，然后取消视频订阅	按 DAG 执行只读依赖，单个 PREPARE_WRITE 节点暂停等待确认

系统架构

后端采用模块化单体，以事务原子性、可审计性和本地部署为优先目标。Vue 3 前端通过 Nginx 访问 Spring Boot REST API。MySQL 保存银行事实、操作单和审计记录；Redis 保存不透明 bearer 会话与短时 AAL 状态。正常前端流量经过 Nginx，Compose 同时把 API、MySQL 和 Redis 端口绑定到本机 loopback，便于本地诊断。

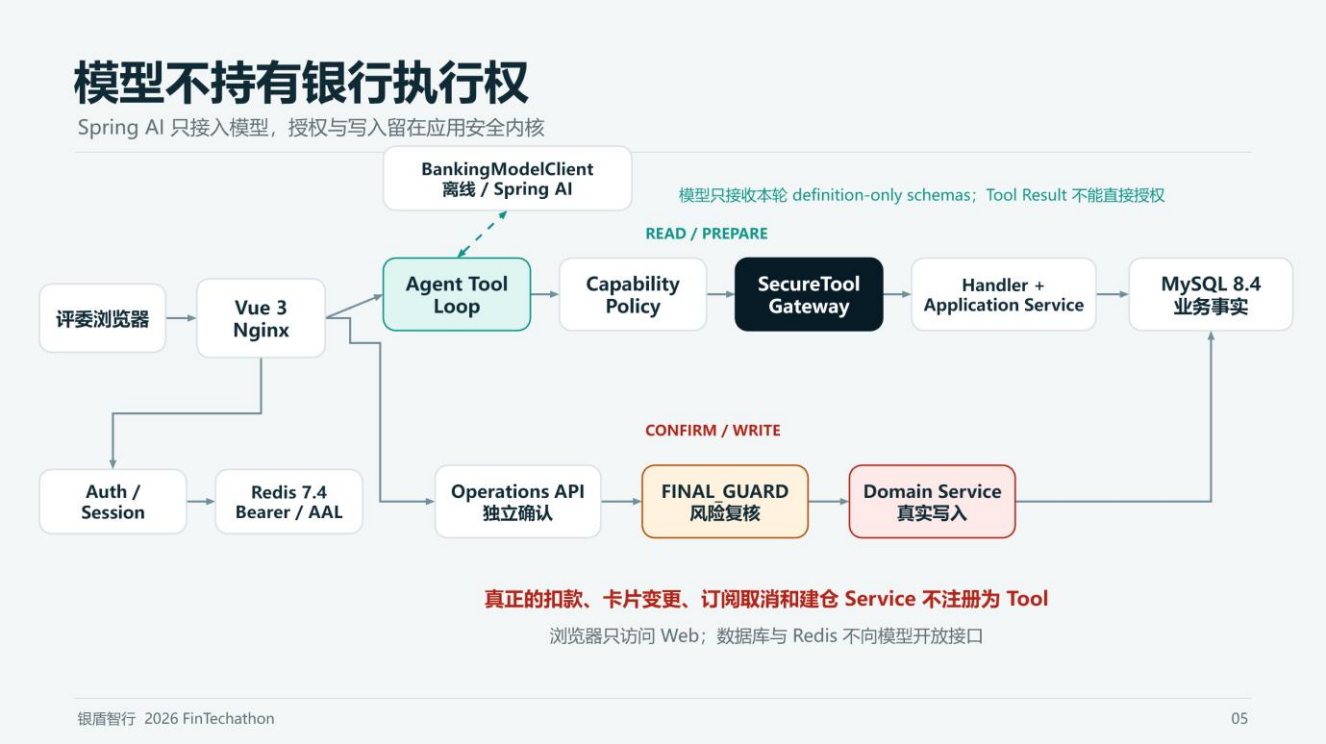


图 1 银盾智行系统架构与主要信任边界

组件职责

组件	职责	禁止事项
Vue 3 与 Nginx	登录、业务页面、Agent 对话、确认入口和 API 反向代理	不根据前端字段决定 userId、角色或 AAL
Agent 编排	意图候选、跨场景计划、受控 Tool Loop、多轮上下文	不访问 Repository、Tool Handler 或 Registry
BankingModelClient	默认确定性离线 Provider；可选 Spring AI OpenAI-compatible 适配器	不自动执行 Tool，不持有数据库连接
Capability Policy	按 Principal 角色与能力、Registry 和候选 Tool 决定成员，并把 AAL 等上下文绑定进短期快照	不接受对话修改策略或身份
SecureToolGateway	精确 Tool 匹配、快照成员复核、参数校验、Handler 路由和审计	门禁失败时不得调用业务 Service
Domain Service	所有权、风险、确认、锁、幂等、记账和状态变化	不接受模型提供的授权事实
MySQL 与 Redis	MySQL 为业务事实源；Redis 保存会话和 AAL	不向模型暴露 SQL、缓存或任意网络 Tool

部署拓扑

```
浏览器 -> Web 容器 Vue 3 + Nginx -> API 容器 Spring Boot
                                    |-> MySQL 8.4
                                    |-> Redis 7.4
                                    |-> BankingModelClient
                                        |-> offline-demo Provider 默认
                                        |-> Spring AI Provider 可选
```

Compose 包含 web、api、mysql 和 redis 四个服务，均配置健康检查。API 镜像使用 Java 17 并以 UID 10001 运行。默认主机绑定地址为 127.0.0.1，演示凭据和 Sandbox reset 能力不应暴露到公网。

Agent 运行机制与核心算法

确定性意图收敛与可插拔模型推理

AgentIntentClassifier 先用确定性规则识别账户、账单、转账、卡片、订阅和理财意图，也识别继续、取消和跨场景表达。AgentToolCandidateSelector 据此生成候选 Tool 集合。默认 OfflineDemoBankingModelClient 使用规则、金额、尾号和已有 Evidence 生成下一步调用；非 offline-demo profile 可以接入 Spring AI ChatModel。

这种设计把能力范围收敛放在模型调用之前。真实模型仍可参与自然语言理解和参数提议，但它不能扩张候选集合、改变 Principal 或绕过 Gateway。当前系统不使用 RAG；银行事实只来自受控 Tool Evidence 或 Java 确定性分析，不从向量库或模型记忆补全。

跨场景任务计划

AgentOrchestrationService 根据已识别场景创建拓扑有序的任务 DAG。计划最多包含 8 个步骤，每个计划最多一个 PREPARE_WRITE 节点。只读节点在依赖满足后执行；写节点准备成功后，计划进入 AWAITING_CONFIRMATION。系统支持槽位补全、暂停、恢复和取消。当前计划状态保存在单 JVM 内存中，这一限制在生产化章节单独列出。

黄金路径	依赖顺序	停止条件
账单与订阅	账户/流水 -> 订阅成本 -> prepare_subscription_cancellation	创建取消操作单后等待独立确认
余额与理财	账户余额 -> 产品 -> 适当性 -> prepare_wealth_purchase	AAL2 与独立确认均满足后才可提交

用户控制的 Tool Calling Loop

```
输入 Principal 与用户消息
选择候选 Tool 并签发 AllowedToolSetSnapshot
只把快照成员转换为 definition-only Tool Schema
调用 BankingModelClient 获取不可信 Tool Call 提议
逐个交给 SecureToolGateway 执行
把 ExecutionEvidence 作为数据回传模型
重复至完成、澄清、拒绝或达到安全上限
GroundedAnswerService 校验最终回答
```

限制	配置值
最大模型轮次	5
每轮最大 Tool 调用	4
总 Tool 调用	12
模型超时	20 秒
Tool 超时	5 秒
用户消息长度	4000 字符
对话上下文	65536 字符
重复调用	相同 name、version 和对象键排序后的参数 JSON 不重复执行

确定性账单与订阅算法

BillingAnalysisService 使用 BigDecimal 计算收入、支出、净额、分类排序、商户频次、前后周期差额和百分比，同时识别金额不低于 1000 元的大额流水。没有数据时返回明确的空样本结果。当前版本没有机器学习异常评分，也没有独立的 recurring detection Tool。

订阅成本分析把 WEEKLY、MONTHLY、QUARTERLY 和 YEARLY 周期统一换算为月度等值，再计算年化和指定月数区间成本。金额由 Java 计算，不交给模型心算。

风险决策与回答校验

风险引擎根据当前数据库事实输出允许、拒绝、需要确认或需要 AAL2，并返回稳定 reason code。资源校验成功后才会创建 operation；风险拒绝会保存 operation、risk、audit 和 idempotency 记录但不生成 challenge，只有允许时才生成 challenge。确认阶段重新读取资源并执行 FINAL_GUARD。GroundedAnswerService 逐项核对数字来源；prepare_* 成功后直接生成确定性的等待确认答复，并用有限规则阻止部分无 PREPARE 的写成功措辞。最终 SUCCEEDED 只由独立 confirm API 的服务端回执证明，不再交给模型判定。

Grounding Guard 不是完整的自然语言蕴含验证。它能约束关键数字并覆盖部分写成功措辞，但仍需真实模型红队测试覆盖更复杂的定性错配。

Tool 与 Capability 模型

项目使用 ExplicitToolRegistry 注册 17 个精确 name 和 version 的 Tool。ToolVisibility 只降低模型噪声，不构成授权边界。AllowedToolSetSnapshot 绑定 snapshotId、requestId、userId、role、AAL、policyVersion、签发与过期时间，并为每个成员记录 name、version、required capability、access mode、schema version、参数名和 descriptor digest。DefaultCapabilityPolicy 用 HMAC-SHA256 对快照签名。

Tool	模式	Capability	结果
get_accounts@1.0	READ_ONLY	ACCOUNT_READ	本人账户列表
get_account_balance@1.0	READ_ONLY	ACCOUNT_READ	本人账户余额
list_transactions@1.0	READ_ONLY	TRANSACTION_READ	本人流水
list_cards@1.0	READ_ONLY	CARD_READ	本人卡片
list_payees@1.0	READ_ONLY	PAYEE_READ	本人收款人
list_subscriptions@1.0	READ_ONLY	SUBSCRIPTION_READ	本人订阅
list_wealth_products@1.0	READ_ONLY	WEALTH_PRODUCT_READ	沙箱理财产品
get_operation_status@1.0	READ_ONLY	OPERATION_READ	本人操作状态
get_card_controls@1.0	READ_ONLY	CARD_READ	卡片控制
analyze_subscription_costs@1.0	READ_ONLY	SUBSCRIPTION_READ	确定性成本分析

Tool	模式	Capability	结果
check_wealth_suitability@1.0	READ_ONLY*	WEALTH_SUITABILITY	适当性评估证据
get_wealth_portfolio@1.0	READ_ONLY	WEALTH_PRODUCT_READ	本人持仓
prepare_transfer@1.0	PREPARE_WRITE	TRANSFER_PREPARE	转账预览
prepare_card_status_change@1.0	PREPARE_WRITE	CARD_MANAGE	卡片状态预览
prepare_card_controls_change@1.0	PREPARE_WRITE	CARD_MANAGE	卡片控制预览
prepare_subscription_cancellation@1.0	PREPARE_WRITE	SUBSCRIPTION_MANAGE	订阅取消预览
prepare_wealth_purchase@1.0	PREPARE_WRITE	WEALTH_PURCHASE	理财购买预览

* check_wealth_suitability 不改变资金、持仓或客户产品状态，但会持久化短期适当性决策证据。因此 READ_ONLY 在这里表示无客户资产副作用，而不是完全不写数据库。

执行端再次校验

1. 严格解析并校验 ToolId，拒绝畸形、未知或版本不匹配的标识。
2. 验证快照 HMAC、策略版本、有效期、requestId、userId、角色和 AAL 上下文。
3. 确认 Tool 位于当前 Allowed Tool Set，并在 ExplicitToolRegistry 精确命中。
4. 比较 descriptor digest、access mode 和 confirmationRequired 策略。
5. 执行严格 JSON 字段、类型、长度、Bean Validation 和业务语义校验。
6. 调用 Handler 和 owner-scoped Application Service；保存最终 ALLOW、DENY 或 ERROR 审计。

Spring AI 侧同时关闭 Tool resolution fallback 和 ChatClient 自动 Tool Calling。每轮使用 definition-only callback，并设置 internalToolExecutionEnabled(false)；若框架尝试执行 callback，代码会直接抛错。

受保护写操作

两阶段提交模型

```
PREPARE  Agent -> Gateway 前置门禁 -> Handler -> Service 所有权/风险
         -> operation + risk/audit/idempotency；仅允许时生成 challenge
CONFIRM  POST /api/v1/operations/{id}/confirm -> owner/challenge/expiry/AAL/hash
         -> lock + FINAL_GUARD -> Domain Service -> Receipt/Audit
```

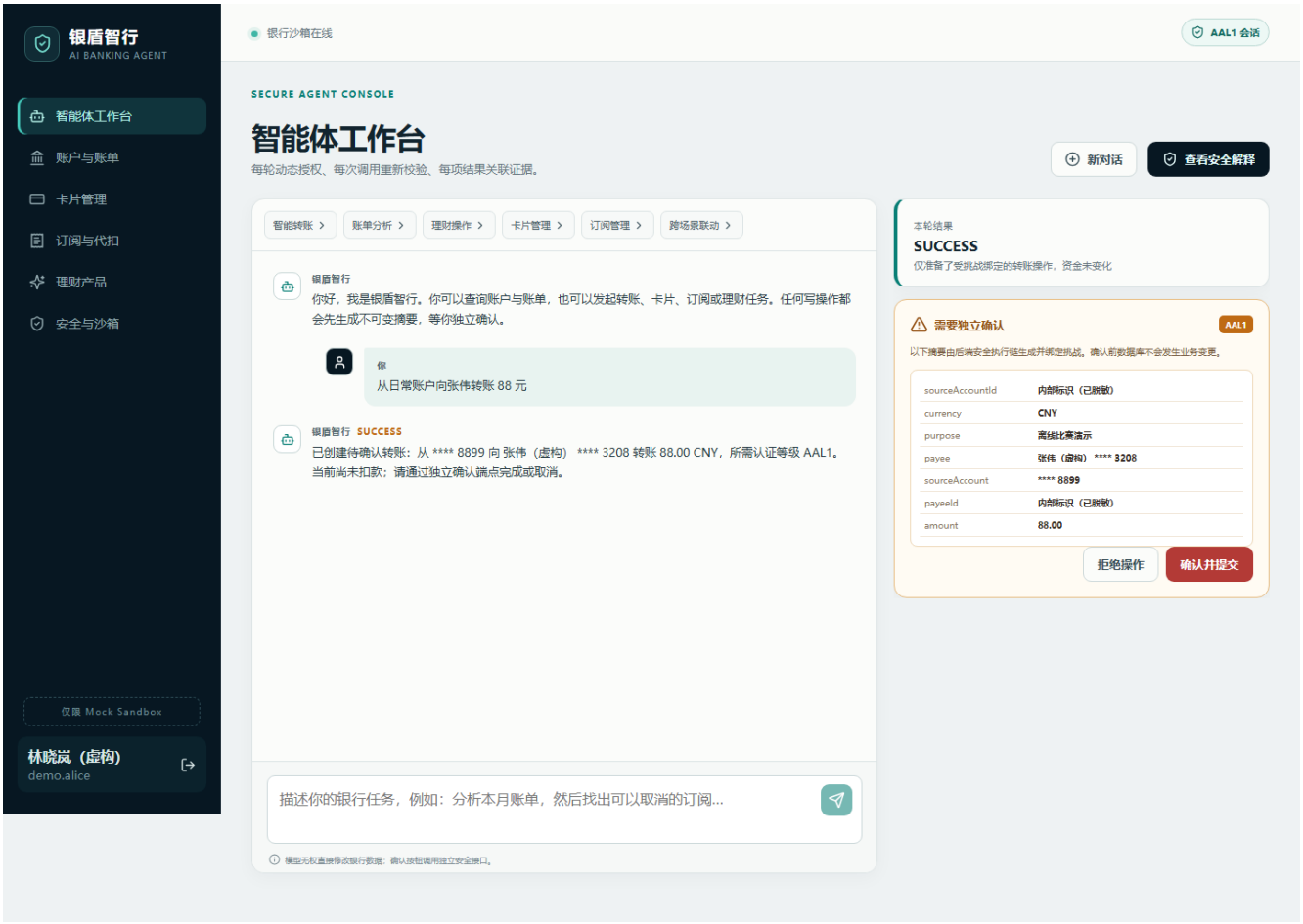


图 2 转账操作准备完成并等待独立确认 此时尚未扣款

领域	准备阶段检查	确认阶段结果
转账	本人 ACTIVE 账户与收款人、币种、余额、日限额、速度；大额或不可信收款人要求 AAL2	重新加锁复核后记账，operation_id 唯一防重复
卡片	合法状态转换和控制 before/after；解冻或放宽需要 AAL2	比较资源版本，写状态或控制变更历史
订阅	本人 ACTIVE 或 PAUSED 且可取消，展示取消影响	比较资源版本，更新状态并写 change history
理财	产品在售、风险匹配、金额、币种、余额和适当性绑定；固定 AAL2	重新校验后扣款、写订单并累计持仓

幂等与并发

幂等键按 userId、业务 scope 和 key hash 建立数据库唯一约束。同一键同一参数返回原 operation 或 receipt；同一键不同参数返回 IDEMPOTENCY_CONFLICT。确认时对 operation 加悲观锁，challenge 只能消费一次，结果表的 operation_id 具有唯一约束。准备与确认均绑定 argsHash 和 summaryHash，防止确认页面与真实参数发生偏差。

安全设计

身份 权限与 AAL

层级	含义	当前能力
未认证（非 AAL 枚举）	无会话	登录、健康检查，以及非 prod 文档/诊断入口
AAL1	用户名密码登录后的普通会话	本人资源查询与创建待确认操作
AAL2	会话内短时加强认证	大额或不可信收款人、理财购买、卡片解冻与放宽控制

角色只有 CUSTOMER 和 DEMO_ADMIN。业务资源始终按服务端 Principal 的 userId 查询；客户端和模型不能提交 userId、role 或 AAL 作为授权字段。他人资源与不存在资源统一返回 404。DEMO_ADMIN 的唯一专属权限是本地 Sandbox reset。prod profile 不注册 reset 组件；step-up 路径仍存在，但 stepUpAvailable=false，会拒绝固定码升级并降级遗留 AAL2 会话。

Prompt Injection 防御

- 系统提示把用户消息、历史和 Tool 文本视为不可信数据，不能改变 Principal、AAL、Capability 或 Allowed Tool Set。
- 模型每轮只看到快照允许的 JSON Schema；隐藏 Tool 不在 schema 中。
- Gateway 在模型之外再次验证快照、成员资格、版本和参数；进入 Handler 后，Application Service 再验证所有权和业务风险，安全性不依赖关键词检测。
- 模型没有 SQL、任意网络、文件或代码执行 Tool，也没有数据库凭据。

- 越权 UUID、附加 userId、instructions、畸形 Tool 名称和不存在版本都在 Handler 前或 owner-scoped Service 中拒绝。

幻觉防护与证据

Tool Result 转换为带 Evidence ID 和摘要哈希的 ExecutionEvidence。关键金额由 Java 计算，回答数字逐项与 Evidence 或确定性分析结果核对。prepare_* 成功后 Agent 直接生成等待确认答复；无 PREPARE 时，有限规则拦截部分写成功措辞。最终状态变化由独立 confirm API 的服务端回执证明，不由模型判定。前端展示本次 Agent 返回的状态、计划原因、Tool 和 Evidence ID；完整数据库审计目前没有独立的公开 explain API。

审计与隐私

Tool Security Audit 保存 request、snapshot、user、Tool、version、decision、reason code、安全说明、JSON 对象键排序后的参数 SHA-256、策略版本、耗时和结果数量，不保存原始参数、Authorization、密码、AAL2 code 或 Prompt。写操作额外保存 operation_request、risk_decision、operation_audit、条件生成的 challenge 和最终回执或变更历史。HTTP 响应使用 X-Trace-Id 关联请求。当前审计表不是 WORM 或外部防篡改日志。

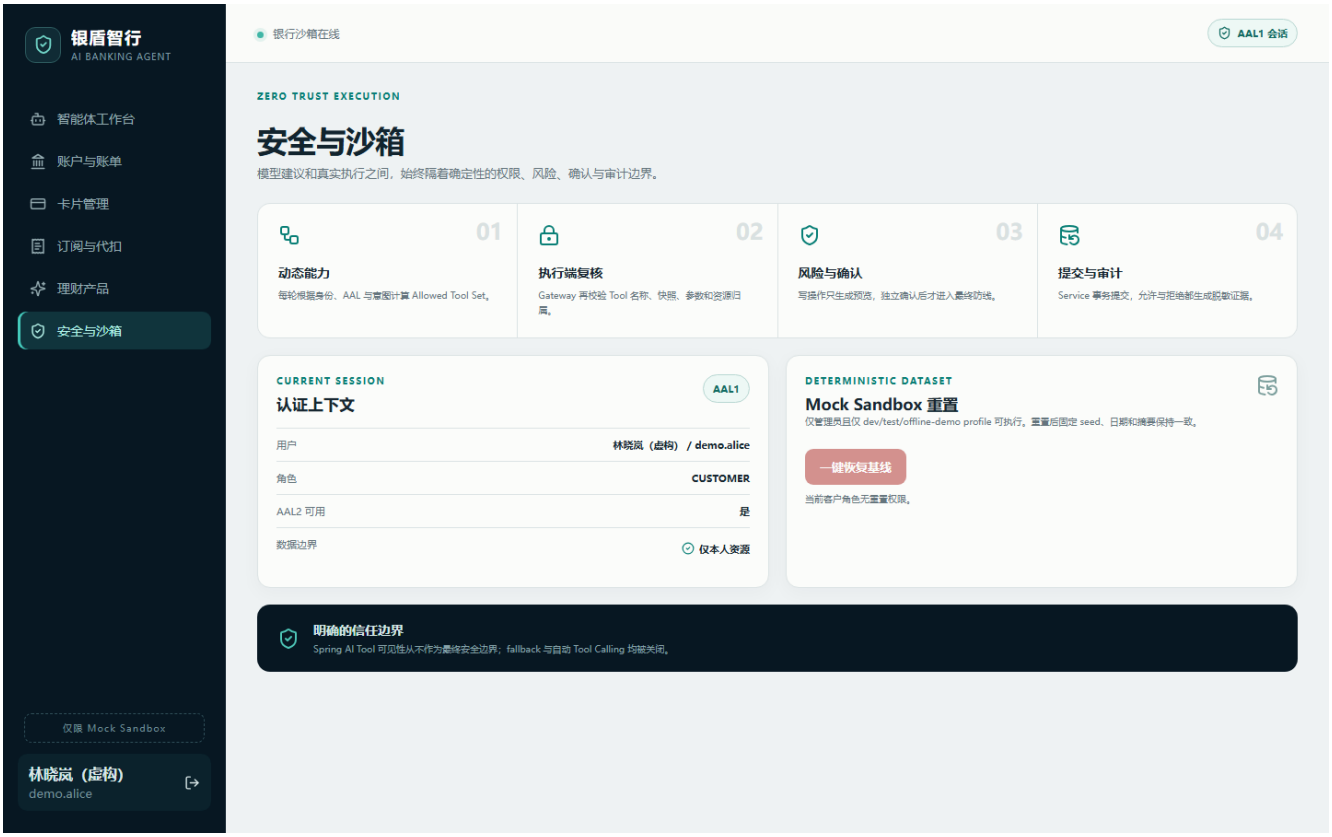


图 3　演示页面展示的安全边界与当前会话

数据库与可复现 Sandbox

Flyway 实际包含 5 个 migration、21 张业务表，另加 flyway_schema_history 共 22 张。表结构按当前迁移文件整理如下。

分组	实际表
身份与数据集	app_user, sandbox_dataset_metadata
核心银行	account, bank_card, payee, account_transaction, subscription, wealth_product
Tool 审计	tool_security_audit
操作内核	operation_request, confirmation_challenge, risk_decision, idempotency_record, transfer_order, operation_audit
理财 卡片 订阅	customer_risk_profile, suitability_check, investment_order, wealth_holding, card_change_history, subscription_change_history

固定项	值
Dataset version	fintechathon-demo-v2
Random seed	2026
Baseline date	2026-08-31
规范化摘要	6c9b472735d704368ae57afe1abb536563b6bd6221404fad0c4f0c986fab65b3
主要演示数据	Alice 2 个账户、2 张卡、5 个收款人、6 个订阅、风险 R3；8 个理财产品

核心 ER 关系

主实体	关系	从实体与用途
app_user	1:N	account、bank_card、payee、subscription；所有客户资源按 user_id 隔离
account	1:N	account_transaction；转账与理财操作在最终提交时锁定并重验账户
operation_request	1:0..1 / 1:N	confirmation_challenge、risk_decision、operation_audit；条件生成 challenge，风险和审计可多阶段记录
wealth_product	1:N	suitability_check、investment_order、wealth_holding；产品版本参与最终绑定复核
idempotency_record	唯一键	user_id + scope + key_hash 唯一，关联原 operation 或最终 receipt

SandboxDatasetGenerator 使用固定 seed 和确定性 UUID 规则生成虚构数据。reset 只在 dev、test、offline-demo profile 注册；prod profile 不创建 reset controller 或 initializer。prod 的 step-up 路径会拒绝固定码升级。连续 reset 后资源内容和规范化摘要一致。

REST API 与前端

分组	接口
认证	POST /auth/login POST /auth/logout GET /auth/me GET /auth/assurance POST /auth/step

分组	接口
本人资源	GET /accounts /accounts/{id} /accounts/{id}/transactions /cards /payees /subscriptions /wealth-products
Agent	POST /agent/chat
Tool 演示	POST /tools/allowed POST /tools/execute
操作	GET /operations/{id} POST /operations/{id}/confirm
Sandbox	GET /sandbox/metadata POST /sandbox/reset 仅非 prod

所有路径前缀为 /api/v1。/tools/allowed 和 /tools/execute 用于比赛诊断与演示；生产化时应移除、内网化或增加渠道与意图策略。前端 token 当前保存在 sessionStorage，适合本地演示，不适合作为生产凭据方案。

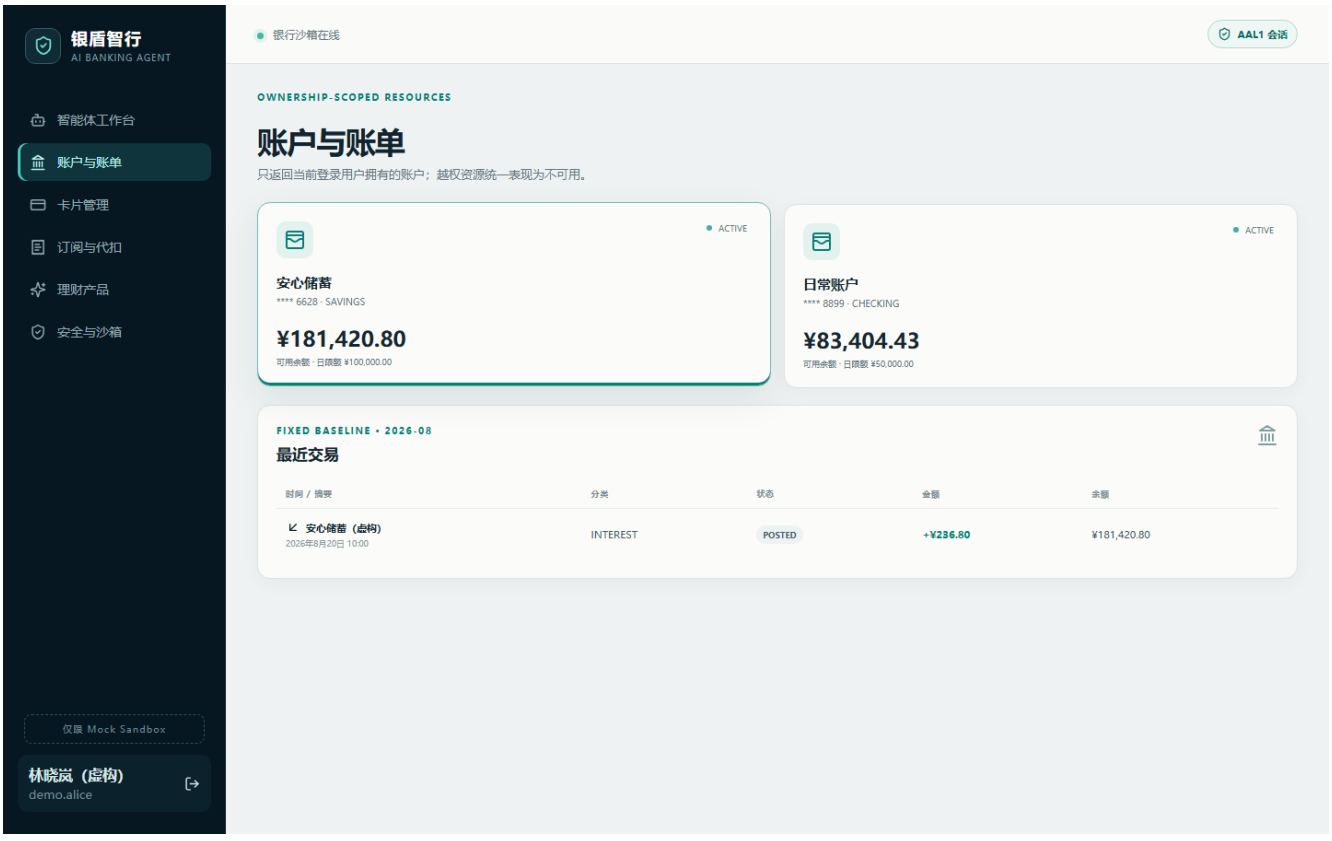


图 4 账户与账单页面只展示当前登录用户的脱敏资源

部署与运行

本地部署需要 Docker Desktop 或 Docker Engine、Docker Compose。默认 offline-demo 无需模型密钥。

```
git clone https://github.com/zhanglonglong01/silver-shield-banking-agent.git
cd silver-shield-banking-agent
docker compose up --build -d --wait
docker compose ps
```

入口	地址
Web	http://127.0.0.1:3000
Swagger UI	http://127.0.0.1:8080/swagger-ui.html
OpenAPI	http://127.0.0.1:8080/v3/api-docs
健康检查	http://127.0.0.1:8080/actuator/health

一键验收使用 scripts/acceptance.ps1。Sandbox reset 使用 scripts/reset-sandbox.ps1。需要接入真实 OpenAI-compatible Provider 时，使用 dev profile 并通过环境变量提供密钥、base URL 和模型名；真实 Provider 不改变 Gateway 的授权边界。

测试与验收

以下结果来自 2026-09-11 的本机最终工作树和交付记录。本次文档制作没有把旧测试数字当作新的运行结果。

门禁	结果	说明
后端 JUnit Testcontainers ArchUnit	162/162 通过	24 suites；MySQL 8.4 与 Redis 7.4 Testcontainers
前端 Vitest	12/12 通过	4 个测试文件
Playwright UI	9/9 通过	使用 page.route 模拟 API，验证 UI、确认卡、AAL2 与安全文本渲染
API 与 Agent 验收	9/9 通过	Docker 真实 API；覆盖固定 reset、越权、PREPARE 零副作用、注入和注销
Compose	4 服务 healthy	API、Web、MySQL、Redis
Flyway	5/5	schema version v005；22 张表含 Flyway history

门禁	结果	说明
并发烟测	100/100	并发 10，客户端 p95 23 ms；不是生产容量结论
依赖故障	通过	Redis 或 MySQL 不可用时 fail closed，恢复后健康

测试覆盖正常、异常、未认证、越权、Prompt Injection、幻觉 Tool 和 ID、错误数字、确认过期、重放、幂等冲突、TOCTOU、并发双确认、Provider 故障与依赖故障。默认 Maven 测试排除 RealLlmIntegrationTest，因此 162 项不代表真实外部模型已经完成验收。

当前边界与生产化工作

- AAL2 使用 Sandbox 固定码，prod 中禁用。生产需要真实 OTP、FIDO、设备绑定或交易签名。
- Compose 使用演示凭据，数据库和 Redis 没有 TLS 或托管密钥，只允许本地 loopback 演示。
- 缺少通用登录和 API 限流、WAF 与账户锁定策略；当前只有 Agent 调用上限和转账速度规则。
- 快照 HMAC key、计划、对话和 Evidence 偏单实例；多实例需要共享状态、密钥轮换与一致性设计。
- 审计保存在普通数据库，不是 WORM 或集中式防篡改日志。
- Grounding Guard 强校验数字并以有限规则覆盖部分写成功措辞，尚未覆盖全部定性语义关系。
- 前端使用 sessionStorage 保存 bearer，生产应改为 BFF 与 HttpOnly、Secure、SameSite Cookie。
- Flyway 对 MySQL 8.4 存在正式测试范围警告；当前 migration 与测试通过，升级前仍需复核。

实现证据索引

主题	仓库证据
Agent Loop	ReadOnlyBankingAgent.java AgentTaskExecutor.java GroundedAnswerService.java
Capability	DefaultCapabilityPolicy.java AllowedToolSetSnapshot.java
Tool 安全	ExplicitToolRegistry.java SecureToolGateway.java ToolArgumentsValidator.java
Spring AI 边界	SpringAiBankingModelClient.java application.yml ToolSafetyConfigurationTest.java
受保护写操作	SecureTransferOperationService.java SecureManagedOperationService.java

主题	仓库证据
架构测试	ArchitectureRulesTest.java
集成与攻击测试	BankingFoundationIntegrationTest.java ReadOnlyBankingAgentTest.java SecureToolGatewayTest.java
部署与运维	compose.yml Dockerfile frontend/Dockerfile scripts/acceptance.ps1